# Especificación de Requisitos de Software – Sistema Web Mil Sabores

Equipo: Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda

## Ficha del documento

| Fecha | Revisión | Autor | Modificación |
|---|---|---|---|
| 2026-10-31 | 0.1 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Introducción y propósito |
| 2026-10-31 | 0.2 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Ámbito del sistema |
| 2026-10-31 | 0.3 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Definiciones, acrónimos y abreviaturas |
| 2026-10-31 | 0.4 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Referencias; se quita DSY1104 del glosario |
| 2026-10-31 | 0.5 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Visión general del documento |
| 2026-11-01 | 0.6 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Descripción general y perspectiva del producto |
| 2026-11-01 | 0.7 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Funciones del producto |
| 2026-11-01 | 0.8 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Características de los usuarios |
| 2026-11-01 | 0.9 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Restricciones |
| 2026-11-01 | 1.0 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Suposiciones y dependencias |
| 2026-11-02 | 1.1 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Requisitos futuros |
| 2026-11-02 | 1.2 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Requisitos comunes de las interfaces |
| 2026-11-02 | 1.3 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Requisitos funcionales: catálogo y carrito (RF-01 |

| Fecha | Revisión | Autor | Modificación |
|---|---|---|---|
| | | | a RF-10) |
| 2026-11-02 | 1.4 | Milton Saavedra, Maximiliano Villalobos, Sebastian Araneda | Se agregan 10 requisitos no funcionales (RNF-01 a RNF-10). |

# 1. Introducción

Este documento especifica los requisitos del Sistema Web Mil Sabores, la tienda online de la Pastelería Mil Sabores, y sigue el estándar IEEE 830. Describe qué debe hacer el sistema, bajo qué restricciones opera y cómo se verificará su cumplimiento.

Pastelería Mil Sabores es una repostería chilena con más de 50 años de trayectoria. El sistema moderniza su canal de venta con una tienda web donde los clientes revisan el catálogo por categorías, aprovechan ofertas, arman un carrito y completan su compra. Incluye además un panel administrativo para gestionar productos, categorías, usuarios y órdenes, y para revisar indicadores de inventario y ventas.

## 1.1. Propósito

El propósito de este documento es definir de forma clara, completa y verificable los requisitos del Sistema Web Mil Sabores. Sirve como base común de entendimiento entre quienes lo solicitan, lo diseñan, lo desarrollan y lo evalúan.

El documento está dirigido a:

- **El cliente (Pastelería Mil Sabores):** para validar que los requisitos reflejan sus necesidades de negocio, como los descuentos por oferta, por edad (clientes mayores de 50 años) y por código promocional (FELICES50).
- **El equipo de pruebas y el docente evaluador:** para planificar y comprobar, con pruebas unitarias y de integración, que el sistema cumple lo especificado.

Cada requisito lleva un código único (por ejemplo RF-01) para asegurar su trazabilidad hacia el diseño, la implementación y las pruebas.

## 1.2. Ámbito del sistema

El sistema se llamará **Sistema Web Mil Sabores**. Es una aplicación web de venta de pasteles con un frontend en React 18 y Bootstrap 5, una API REST en Spring Boot 3 (Java 17) con Spring Data JPA, y una base de datos SQL (MySQL).

**Qué hará el sistema**

- Mostrar el catálogo de productos y filtrarlo por categoría (Tortas Cuadradas, Tortas Circulares, Sin Azúcar y Sin Gluten), con una vista de ofertas y una de detalle de producto.
- Gestionar un carrito de compras que respeta el stock disponible y conserva su contenido entre visitas.

- Calcular precios y descuentos: el descuento de cada producto en oferta, 50 % para clientes mayores de 50 años y 10 % vitalicio con el código FELICES50. Los descuentos de cliente no se acumulan; se aplica el mayor.
- Registrar e identificar usuarios. El servidor cifra la contraseña con BCrypt y calcula los beneficios del cliente.
- Completar la compra con un formulario de despacho validado y una pasarela de pago simulada, que lleva a una pantalla de éxito con número de orden o a una de reintento.
- Ofrecer un panel de administración, protegido por rol, para gestionar productos, categorías y usuarios, y para consultar órdenes, boletas, productos con stock crítico (menos de 6 unidades) y reportes de ventas.
- Exponer una API REST (`/api/productos`, `/api/categorias`, `/api/usuarios` y `/api/ordenes`) con CORS habilitado para el frontend.
- Mantener el catálogo, las categorías, el carrito y las órdenes en el navegador (localStorage) cuando el backend no responde.
- Incluir pruebas unitarias automatizadas con Vitest sobre renderizado, eventos, reglas de negocio y persistencia.

**Qué no hará el sistema**

- Procesar pagos reales ni conectarse a una pasarela bancaria.
- Gestionar el despacho ni el seguimiento de los pedidos.
- Descontar automáticamente el stock al confirmar una orden.
- Autenticar con tokens ni autorizar por rol en la API; hoy el acceso al panel se restringe solo en el frontend.
- Permitir iniciar sesión o registrarse sin el backend en ejecución.
- Enviar correos reales desde el formulario de contacto.

**Beneficios, objetivos y metas**

- Ampliar el canal de venta de la pastelería con una tienda disponible en cualquier momento y adaptada a pantallas móviles.
- Aplicar de forma automática y consistente las reglas de descuento, con el cálculo de beneficios hecho en el servidor y no en el navegador.
- Darle a la administración un único lugar para controlar el catálogo, el inventario crítico y las ventas.
- Mantener la tienda usable para consultar el catálogo aunque el servidor esté caído.
- Asegurar la calidad con un conjunto de pruebas unitarias repetible con el comando `npm test`.

## 1.3. Definiciones, acrónimos y abreviaturas

Esta sección reúne los términos técnicos y de negocio que se usan en el documento, para que cada requisito tenga una sola interpretación.

| Término | Definición |
|---|---|
| ERS | Especificación de Requisitos de Software. |

| Término | Definición |
|---|---|
| IEEE 830 | Estándar que define la estructura y las características de una buena ERS. |
| RF | Requisito funcional, identificado con un código único (RF-01, RF-02, etc.). |
| Frontend | Parte del sistema que ve el usuario en el navegador, hecha con React y Bootstrap. |
| Backend | Parte del sistema que corre en el servidor, hecha con Spring Boot. Valida datos, aplica reglas y guarda en la base de datos. |
| API REST | Interfaz por la que el frontend pide y envía datos al backend mediante HTTP y JSON. |
| CRUD | Crear, leer, actualizar y eliminar registros (por ejemplo, productos). |
| CORS | Mecanismo que permite que el frontend, en otro origen, consuma la API del backend. |
| JPA | Especificación de Java para guardar objetos en una base de datos relacional. Se usa mediante Spring Data JPA. |
| SQL / MySQL | Lenguaje de consultas y motor de base de datos relacional donde se guardan productos, categorías, usuarios y órdenes. |
| BCrypt | Algoritmo de cifrado que se aplica a las contraseñas antes de guardarlas, de modo que no se almacenan en texto plano. |
| localStorage | Almacenamiento del navegador que guarda el carrito, la sesión y el respaldo local del catálogo. |
| Respaldo local | Copia en localStorage del catálogo, las categorías y las órdenes, usada cuando el backend no responde. |
| Vitest | Herramienta de pruebas unitarias del frontend, ejecutada con `npm test`. |
| Producto | Artículo del catálogo con nombre, categoría, precio, stock, descripción, imagen y datos de oferta. |
| Categoría | Grupo de productos: Tortas Cuadradas, Tortas Circulares, Sin Azúcar o Sin Gluten. |
| Stock | Unidades disponibles de un producto. Un carrito no puede superarlo. |
| Stock crítico | Situación de un producto con menos de 6 unidades. |

| Término | Definición |
|---|---|
| Oferta | Descuento porcentual propio de un producto, que se aplica a su precio unitario. |
| Carrito | Lista de productos y cantidades que el cliente reúne antes de pagar. |
| Orden | Registro de una compra confirmada, con cliente, dirección, ítems, subtotal, descuento y total. Su identificador tiene la forma ORD-seguido de números. |
| Boleta | Comprobante de una orden que el administrador consulta por su identificador. |
| Pasarela de pago simulada | Paso de pago que no cobra dinero y permite elegir si la transacción termina en éxito o en rechazo. |
| Cliente | Usuario con rol cliente que compra en la tienda. |
| Administrador | Usuario con rol admin que accede al panel de gestión. |
| Beneficio | Descuento de cliente asignado al registrarse: MAYOR_50 o FELICES50. |
| MAYOR_50 | Beneficio de 50 % de descuento para clientes de más de 50 años. |
| FELICES50 | Código promocional que otorga 10 % de descuento vitalicio al registrarse. |

### 1.4. Referencias

Estos son los documentos y fuentes en los que se basa esta ERS.

| Código | Documento | Uso en esta ERS |
|---|---|---|
| REF-01 | IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications | Estructura y características de los requisitos. |
| REF-02 | DSY1104 Evaluación Parcial 1 – Anexo 4: ERS, Especificación de Requisitos del Software (Duoc UC) | Plantilla de las secciones de este documento. |
| REF-03 | ERS V2 – Pastelería Mil Sabores (docs/ERS_V2_Mil_Sabores.md) | Requisitos funcionales RF-01 a RF-08 del proyecto. |
| REF-04 | Documento de Cobertura de Testing (Vitest) (docs/Cobertura_Testing_Vitest.md) | Pruebas unitarias del frontend y su comportamiento esperado. |

| Código | Documento | Uso en esta ERS |
|---|---|---|
| REF-05 | README del proyecto (README.md) | Instrucciones de ejecución, usuario administrador de prueba y reglas de descuento. |
| REF-06 | Código fuente del proyecto (carpetas frontend y backend) | Arquitectura actual, endpoints de la API y modelo de datos. |

### 1.5. Visión general del documento

El documento tiene tres secciones: una de contexto, una de descripción del producto y una con los requisitos detallados.

- **Sección 1, Introducción:** presenta el propósito, el ámbito del sistema, las definiciones, las referencias y esta visión general.
- **Sección 2, Descripción general:** explica el contexto del producto y no los requisitos. Cubre la perspectiva del producto, sus funciones, los tipos de usuario (cliente y administrador), las restricciones, las suposiciones y dependencias, y los requisitos futuros.
- **Sección 3, Requisitos específicos:** detalla lo que el sistema debe cumplir. Incluye las interfaces de usuario, de hardware, de software y de comunicación, los requisitos funcionales (RF) y los no funcionales (rendimiento, seguridad, fiabilidad, disponibilidad, mantenibilidad y portabilidad), además de otros requisitos.

Cada requisito tiene un código único para poder rastrearlo hasta el diseño, el código y las pruebas. La ficha del documento registra cada revisión; la versión sube 0.1 con cada sección que se agrega.

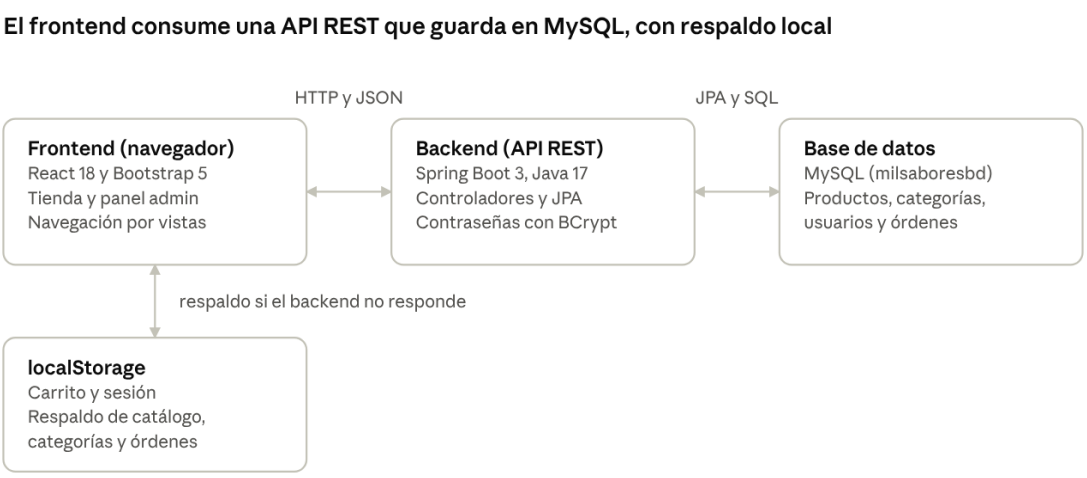
## 2. Descripción general

Esta sección explica el contexto del Sistema Web Mil Sabores. No define requisitos; eso se hace en la sección 3.

### 2.1. Perspectiva del producto

El sistema es un producto independiente. No forma parte de un sistema mayor ni se integra con sistemas externos, como una pasarela de pago real o un servicio de correo. Se compone de tres partes que se comunican entre sí:

- **Frontend:** una aplicación React 18 con Bootstrap 5 que corre en el navegador. Atiende la tienda y el panel de administración, y cambia de vista según el estado de la aplicación.
- **Backend:** una API REST en Spring Boot 3 (Java 17) con Spring Data JPA. Expone los endpoints `/api/productos`, `/api/categorias`, `/api/usuarios` y `/api/ordenes`, cifra las contraseñas con BCrypt y calcula los beneficios de cada cliente.
- **Base de datos:** MySQL, donde se guardan productos, categorías, usuarios y órdenes.




arquitectura del sistema · 4 componentes

El frontend habla con el backend por HTTP, enviando y recibiendo JSON. Si el backend no responde, el catálogo, las categorías y las órdenes siguen funcionando con un respaldo en `localStorage`; el inicio de sesión y el registro, en cambio, requieren el backend. Las órdenes confirmadas también se copian en `localStorage`, y de esa copia leen el historial de compras, la boleta y los reportes del panel de administración.

Para funcionar, la interfaz carga recursos externos de solo lectura: Bootstrap y las fuentes desde CDN, y las imágenes de los productos desde la URL de cada uno.

## 2.2. Funciones del producto

El sistema agrupa sus funciones en seis módulos. Aquí se resumen a grandes rasgos; el detalle de cada función está en los requisitos de la sección 3.

| Módulo | Funciones principales | Usuario |
|---|---|---|
| Catálogo | Listar productos, filtrarlos por categoría, ver las ofertas y abrir el detalle de un producto. | Visitante y cliente |
| Carrito | Agregar productos, cambiar cantidades sin superar el stock, eliminar con confirmación y ver el resumen con ahorro por ofertas, descuento de cliente y total. | Visitante y cliente |
| Usuarios y sesión | Registrarse con fecha de nacimiento y código promocional opcional, iniciar y cerrar sesión, y recibir los beneficios que correspondan. | Visitante y cliente |
| Compra | Completar el formulario de despacho con validaciones, pasar por la pasarela de pago simulada y ver la confirmación con el número | Cliente |

| Módulo | Funciones principales | Usuario |
|---|---|---|
| | de orden o la pantalla de reintento. | |
| Contenido informativo | Consultar las secciones Nosotros, Blog y Contacto. | Visitante y cliente |
| Panel de administración | Gestionar productos (crear, editar y eliminar) y categorías (crear y eliminar), listar usuarios, ver productos con stock crítico, revisar el historial de compras, buscar una boleta por su número y ver reportes de órdenes e ingresos. | Administrador |

Todas las funciones del frontend se apoyan en la API REST del backend, que valida los datos, aplica las reglas de negocio y guarda la información en la base de datos.

### 2.3. Características de los usuarios

El sistema tiene tres perfiles de usuario. Ninguno necesita conocimientos técnicos de programación.

| Perfil | Descripción | Qué puede hacer | Conocimientos requeridos |
|---|---|---|---|
| Visitante | Persona que entra a la tienda sin iniciar sesión. | Ver el catálogo, las ofertas y el detalle de los productos, llenar el carrito, leer el blog y escribir desde el formulario de contacto, y crear una cuenta o iniciar sesión. | Uso básico de internet y de un navegador, en computador o celular. |
| Cliente | Persona registrada, con rol cliente. Puede tener beneficios de descuento (mayor de 50 años o código FELICES50). | Todo lo del visitante, más comprar con sus datos autocompletados y recibir sus descuentos en el carrito. | Los mismos que el visitante. |
| Administrador | Persona de la pastelería con rol admin, a cargo de la gestión de la tienda. | Entrar al panel de administración y gestionar productos, categorías, usuarios, órdenes y reportes. | Uso básico de computador y navegador, y conocimiento del inventario y los precios de la pastelería. |

Los clientes se registran por su cuenta y siempre quedan con el rol cliente. La cuenta de administrador existe desde la carga inicial de datos del sistema.

### 2.4. Restricciones

Estas son las limitaciones que condicionan el desarrollo y el uso del sistema.

| Categoría | Restricción |
|---|---|
| Lenguajes y tecnologías | El frontend se escribe en JavaScript con React 18 y Bootstrap 5. El backend usa Java 17, Spring Boot 3.2.4 y Spring Data JPA. La base de datos es MySQL. Las pruebas unitarias usan Vitest, que requiere Node.js 20.19 o superior (o 22.12 o superior). |
| Protocolos de comunicación | El frontend y el backend se comunican por HTTP con datos en formato JSON. |
| Interfaces con otras aplicaciones | El sistema no se integra con aplicaciones externas. Solo carga recursos de lectura: Bootstrap y fuentes desde CDN, e imágenes de productos desde su URL. |
| Dispositivos | Requiere un navegador web moderno con JavaScript y `localStorage` habilitados. La interfaz se adapta a pantallas de computador y de celular. |
| Persistencia | La base de datos del backend es la fuente de verdad. El respaldo en `localStorage` solo cubre catálogo, categorías, carrito y órdenes cuando el backend no responde. El registro y el inicio de sesión siempre necesitan el backend. |
| Reglas de negocio | Los descuentos de cliente no se acumulan: se aplica el mayor. El beneficio por edad solo corresponde a clientes con más de 50 años. El código FELICES50 no distingue mayúsculas de minúsculas. El carrito no puede superar el stock. El correo de cada usuario y el nombre de cada categoría deben ser únicos. |
| Validación de datos | La contraseña debe tener al menos 6 caracteres. La fecha de nacimiento es obligatoria y no puede ser futura. Al guardar un producto, el precio debe ser mayor que 0, el stock un entero desde 0 y el descuento de oferta estar entre 1 y 99. |
| Pagos | La pasarela de pago es simulada y no cobra dinero. |
| Auditoría | El sistema no registra qué cambios hace cada administrador. |
| Criticidad | Es un proyecto académico (asignatura DSY1104) que no maneja pagos ni datos bancarios reales. |

**Limitaciones de seguridad actuales**

El sistema aplica estas medidas: las contraseñas se guardan cifradas con BCrypt, la API nunca las devuelve y el navegador no las almacena. Además, el servidor fuerza el rol cliente en todo registro nuevo. Aun así, estas limitaciones siguen vigentes:

- La API no autentica ni autoriza por rol: cualquier cliente HTTP puede usar sus endpoints. El acceso al panel de administración se restringe solo en el frontend.
- El botón "Acceso Rápido Administrador" de la pantalla de inicio de sesión entra al panel sin contraseña. Es solo para demostración.
- El backend acepta peticiones desde cualquier origen (CORS abierto).
- Las credenciales de la base de datos están escritas en el archivo `application.properties`.

## 2.5. Suposiciones y dependencias

Si alguno de estos factores cambia, habría que revisar los requisitos de la sección 3.

**Suposiciones**

- El backend y MySQL están en ejecución. Sin ellos no se puede registrar ni iniciar sesión, y los datos no se guardan de forma permanente.
- MySQL corre en `localhost:3306` y el backend crea la base `milsaboresbd` si no existe.
- El frontend busca el backend en `http://localhost:9091/api`. Si está en otra dirección, se cambia con la variable `REACT_APP_API_URL`.
- La carga inicial de datos deja listas las categorías, los productos de ejemplo y la cuenta de administrador.
- El navegador permite usar `localStorage`. Si se borra o se navega en modo privado, el carrito, la sesión y el respaldo local se pierden.
- Las reglas de descuento no cambian: 50 % para mayores de 50 años y 10 % con el código FELICES50.

**Dependencias**

| Dependencia | Para qué se usa | Si falta |
|---|---|---|
| JDK 17 o 21 y Maven 3.8 o superior | Compilar y ejecutar el backend. | El backend no arranca. |
| Node.js 20.19 o superior (o 22.12 o superior) y npm | Ejecutar el frontend y las pruebas con Vitest. | No se puede levantar el frontend ni correr las pruebas. |
| MySQL 8 | Guardar productos, categorías, usuarios y órdenes. | El backend no arranca y el frontend usa el respaldo local. |
| Spring Boot, Spring Data JPA, spring-security-crypto (BCrypt) y el conector de MySQL | Construir la API, acceder a los datos y cifrar contraseñas. | El backend no compila. |
| React, Bootstrap, bootstrap-icons y react-scripts | Construir la interfaz. | El frontend no compila. |

| Dependencia | Para qué se usa | Si falta |
|---|---|---|
| CDN de Bootstrap y Google Fonts | Cargar estilos y tipografías en `index.html`. | La interfaz pierde parte de su aspecto. |
| Imágenes alojadas en Unsplash y en el sitio elingenio.cl | Mostrar las fotos de los productos. | Las fotos no se ven. |

### 2.6. Requisitos futuros

Estas mejoras no forman parte de la versión actual. Se podrán analizar e implementar más adelante y, en su mayoría, resuelven las limitaciones descritas en las secciones 1.2 y 2.4.

| Mejora | Qué resolvería |
|---|---|
| Autenticación con tokens y autorización por rol en la API | Hoy cualquier cliente HTTP puede usar los endpoints; la protección del panel de administración existe solo en el frontend. |
| Quitar el acceso rápido de administrador | Hoy permite entrar al panel sin contraseña. |
| Descontar el stock al confirmar una orden | Hoy el stock solo cambia cuando el administrador lo edita. |
| Leer las órdenes del panel desde la base de datos | Hoy el historial, la boleta y los reportes usan la copia local del navegador, así que solo muestran las compras hechas en ese equipo. |
| Dashboard resumen con datos reales | Hoy muestra cifras de ejemplo cuando los contadores están en cero. |
| Pasarela de pago real | Hoy la pasarela es simulada y no cobra. |
| Historial de compras para cada cliente y seguimiento del despacho | Hoy solo el administrador ve las órdenes y no hay estados de pedido. |
| Envío real de los mensajes del formulario de contacto | Hoy el formulario solo confirma en pantalla. |
| Gestión completa de usuarios en el panel (cambiar rol, desactivar cuentas) | Hoy el panel solo lista los usuarios. |
| Imágenes propias y carga de imágenes desde el panel | Hoy las fotos dependen de sitios de terceros. |
| Registro de auditoría de los cambios del administrador | Hoy no queda constancia de quién cambió qué. |
| Credenciales por variables de entorno | Hoy la contraseña de la base de datos está escrita en `application.properties`. |

## 3. Requisitos específicos

Esta sección detalla lo que el sistema debe cumplir, con el nivel de precisión necesario para diseñarlo y probarlo. Cada requisito tiene un código único, para poder rastrearlo hasta el diseño, el código y las pruebas.

### 3.1. Requisitos comunes de las interfaces

#### 3.1.1. Interfaces de usuario

La interfaz es una aplicación web con un menú superior, un área central de contenido y un pie de página. El menú superior muestra Inicio, Categorías, Ofertas, Nosotros, Blog y Contacto, el carrito con su cantidad y total, y las opciones de sesión: Ingresar y Registrarse, o el nombre del usuario con el botón Salir. El administrador ve además la opción Admin Panel.

El estilo usa Bootstrap 5 con tonos café sobre un fondo crema. Los textos van en la tipografía Lato y el nombre de la marca en Pacifico. Los productos se muestran en tarjetas con imagen, categoría, insignia de descuento cuando están en oferta, precio original tachado junto al precio final y el botón Añadir. Los errores de los formularios se muestran en alertas sobre el formulario. La interfaz se adapta al ancho de la pantalla, y el menú se colapsa en pantallas pequeñas.

| Pantalla | Contenido | Acceso |
|---|---|---|
| Inicio | Banner de aniversario y catálogo destacado. | Todos |
| Categorías | Catálogo con botones para filtrar por categoría. | Todos |
| Ofertas | Productos que están en oferta. | Todos |
| Detalle de producto | Imagen, descripción, precio con descuento, stock y botón para agregar al carrito. | Todos |
| Carrito | Tabla de productos con cantidades y resumen con ahorro, descuento de cliente y total. | Todos |
| Checkout | Formulario de despacho, opción para simular un error de pago y botón de pago. | Todos |
| Pago exitoso y pago rechazado | Número de orden y total pagado, o aviso de rechazo con botón para reintentar. | Todos |
| Iniciar sesión y Crear cuenta | Formularios de acceso y de registro. | Todos |
| Nosotros, Blog, Detalle de blog y Contacto | Contenido informativo y formulario de contacto. | Todos |
| Gestión operativa | Boletas, productos críticos, categorías, reportes, usuarios, historial de compras y gestión de productos. | Administrador |
| Dashboard resumen | Tarjetas con compras, productos y usuarios, y | Administrador |

| Pantalla | Contenido | Acceso |
|---|---|---|
| | accesos rápidos a los módulos. | |

### 3.1.2. Interfaces de hardware

El sistema no requiere periféricos ni dispositivos especiales. Se usa desde un navegador en computador, tableta o celular, y su interfaz adaptable permite manejarla con pantalla táctil.

### 3.1.3. Interfaces de software

| Producto | Propósito | Definición de la interfaz |
|---|---|---|
| MySQL 8 | Guardar los datos del sistema. | El backend accede con Spring Data JPA y el conector JDBC de MySQL. Las tablas son `productos`, `categorias`, `usuarios`, `usuario_beneficios`, `ordenes` y `orden_items`. |
| `localStorage` del navegador | Guardar el carrito, la sesión y el respaldo local. | Valores en formato JSON bajo las claves `pms_cart`, `pms_session`, `pms_productos`, `pms_categorias` y `pms_ordenes`. |
| CDN de Bootstrap y Google Fonts | Cargar estilos, scripts y tipografías. | Archivos CSS y JavaScript referenciados en `index.html`. |
| Imágenes de productos | Mostrar las fotos del catálogo. | URL de la imagen guardada en cada producto. |

### 3.1.4. Interfaces de comunicación

El frontend y el backend se comunican por HTTP con datos en JSON. La URL base de la API es `http://localhost:9091/api` y se puede cambiar con la variable `REACT_APP_API_URL`. El backend habilita CORS para las rutas `/api/**` con los métodos GET, POST, PUT, DELETE y OPTIONS.

| Endpoint | Descripción | Respuestas |
|---|---|---|
| GET `/api/productos` | Lista los productos. | 200 |
| GET `/api/productos/{id}` | Entrega un producto. | 200, 404 |
| POST `/api/productos` | Crea un producto. | 201 |
| PUT `/api/productos/{id}` | Actualiza un producto. | 200, 404 |
| DELETE `/api/productos/{id}` | Elimina un producto. | 204, 404 |
| GET `/api/categorias` | Lista las categorías. | 200 |
| POST `/api/categorias` | Crea una categoría. | 201, 400 si falta el nombre, 409 si ya existe |

| Endpoint | Descripción | Respuestas |
|---|---|---|
| DELETE `/api/categorias/{id}` | Elimina una categoría. | 204, 404 |
| GET `/api/usuarios` | Lista los usuarios, sin contraseñas. | 200 |
| POST `/api/usuarios/registro` | Registra un cliente y calcula sus beneficios. | 201, 400 si los datos son inválidos, 409 si el correo ya existe |
| POST `/api/usuarios/login` | Valida las credenciales y entrega el usuario, sin contraseña. | 200, 401 |
| GET `/api/ordenes` | Lista las órdenes. | 200 |
| GET `/api/ordenes/{id}` | Entrega una orden. | 200, 404 |
| POST `/api/ordenes` | Registra una orden con sus ítems. | 201, 400 si el total falta o es negativo |

## 3.2. Requisitos funcionales

Los requisitos funcionales se numeran de RF-01 a RF-29 y se agrupan por módulo. Cada uno indica su actor, su prioridad y, cuando existe, el requisito equivalente del ERS V2 (REF-03). La prioridad **Esencial** señala lo que el sistema necesita para cumplir su propósito; **Deseable** señala lo que mejora la experiencia sin ser indispensable.

### 3.2.1. Catálogo

| Código | Requisito | Actor | Prioridad | ERS V2 |
|---|---|---|---|---|
| RF-01 | **Listar el catálogo.** El sistema debe mostrar los productos en tarjetas con imagen, nombre, categoría, descripción y precio. Los datos vienen de la API; si el backend no responde, vienen del respaldo local. | Visitante y cliente | Esencial | RF-01 |
| RF-02 | **Filtrar por categoría.** El sistema debe ofrecer un botón por cada categoría, más el botón Todas, y mostrar solo los productos de la categoría elegida. | Visitante y cliente | Esencial | RF-02 |
| RF-03 | **Ver las ofertas.** El sistema debe tener una vista que muestre solo los productos en oferta. | Visitante y cliente | Deseable | RF-01 |
| RF-04 | **Ver el detalle de un producto.** Al hacer clic en la imagen o en el nombre de una tarjeta, el sistema debe mostrar imagen, categoría, nombre, descripción, precio, stock disponible y un botón para agregar al carrito. El botón se deshabilita si el stock es 0. Si el producto no existe, el sistema debe | Visitante y cliente | Esencial | RF-01 |

| Código | Requisito | Actor | Prioridad | ERS V2 |
|---|---|---|---|---|
| | mostrar el aviso "Producto no encontrado". | | | |
| RF-05 | **Calcular el precio final.** Si el producto está en oferta con un descuento mayor que 0, el precio final es el precio por (1 − descuento / 100), redondeado al entero más cercano. En otro caso, es el precio normal. Cuando hay descuento, el sistema debe mostrar el precio original tachado, el precio final y una insignia con el porcentaje. Los precios se muestran en pesos chilenos con separador de miles. | Visitante y cliente | Esencial | RF-01 |

**3.2.2. Carrito de compras**

| Código | Requisito | Actor | Prioridad | ERS V2 |
|---|---|---|---|---|
| RF-06 | **Agregar al carrito.** Al presionar Añadir, el sistema debe agregar una unidad del producto. Si ya está en el carrito, debe aumentar su cantidad sin superar el stock. No debe agregar productos sin stock. | Visitante y cliente | Esencial | RF-03 |
| RF-07 | **Cambiar la cantidad.** El sistema debe permitir subir o bajar la cantidad de cada producto. El botón para subir se deshabilita al llegar al stock. Si la cantidad baja a 0, el sistema debe pedir confirmación para eliminar el producto. | Visitante y cliente | Esencial | RF-03 |
| RF-08 | **Eliminar un producto.** El sistema debe pedir confirmación, nombrando el producto, antes de quitarlo del carrito. | Visitante y cliente | Esencial | RF-03 |
| RF-09 | **Mostrar el resumen del carrito.** El sistema debe calcular: subtotal (suma del precio final por la cantidad de cada producto), ahorro por ofertas, descuento de cliente y total. El descuento de cliente es el subtotal por el porcentaje del beneficio, redondeado, y el total es el subtotal menos ese descuento. El ahorro y el descuento solo se muestran si son mayores que 0. Si no hay sesión iniciada, el sistema debe avisar que hay que iniciar sesión para aplicar los beneficios. Si el carrito está vacío, debe mostrar un aviso y un botón hacia el catálogo. | Visitante y cliente | Esencial | RF-03 |
| RF-10 | **Conservar el carrito.** El sistema debe guardar el carrito en el navegador y restaurarlo al volver a abrir la tienda. El menú superior debe mostrar el total de | Visitante y cliente | Deseable | RF-03 |

| Código | Requisito | Actor | Prioridad | ERS V2 |
|---|---|---|---|---|
| | unidades y el total a pagar. | | | |

## 3.3. Requisitos no funcionales

Los requisitos no funcionales establecen criterios de calidad y condiciones técnicas que debe cumplir el Sistema Web Mil Sabores. Se definen considerando una aplicación fullstack desarrollada con React, Bootstrap, Spring Boot y una base de datos MySQL utilizada en un entorno de desarrollo y pruebas. Los datos de ejemplo o simulados no deben confundirse con información de producción.

| **Código** | **Requisito no funcional** | **Categoría** | **Prioridad** |
|---|---|---|---|
| RNF-01 | Rendimiento de la interfaz. El sistema debe mostrar las vistas principales (Inicio, Categorías, Ofertas y Carrito) en un máximo de 3 segundos, en un equipo de prueba estándar y con una conexión estable. | Rendimiento | Esencial |
| RNF-02 | Tiempo de respuesta de la API. En el entorno de pruebas, al menos el 95 % de las solicitudes habituales de consulta (GET) a la API REST debe responder en un máximo de 2 segundos, con hasta 20 usuarios concurrentes. | Rendimiento | Deseable |
| RNF-03 | Diseño adaptable. La interfaz debe poder utilizarse sin pérdida de contenido ni funciones principales en anchos de pantalla desde 360 px (móvil) hasta 1920 px (escritorio). | Usabilidad | Esencial |
| RNF-04 | Compatibilidad con navegadores. La aplicación debe funcionar correctamente en las dos versiones estables más recientes de Google Chrome, Mozilla Firefox y Microsoft Edge, con JavaScript y localStorage habilitados. | Compatibilidad | Esencial |
| RNF-05 | Facilidad de uso. Las opciones principales de navegación y las acciones de compra (ver producto, agregar al carrito y acceder al checkout) deben estar identificadas con textos claros y ser alcanzables desde la navegación principal en un máximo de 3 interacciones. | Usabilidad | Deseable |
| RNF-06 | Protección de datos de prueba. El sistema no debe utilizar datos bancarios reales ni almacenar contraseñas en texto plano. Las contraseñas de las cuentas de prueba deben almacenarse mediante BCrypt y nunca deben incluirse en las respuestas de la API ni en localStorage. | Seguridad | Esencial |
| RNF-07 | Integridad y persistencia de datos. Las operaciones de creación, consulta, modificación y eliminación deben conservar la consistencia de los registros entre el backend y la base de datos de desarrollo. Cuando se utilicen datos simulados, estos deben estar identificados como datos de prueba y no presentarse como transacciones reales. | Integridad | Esencial |
| RNF-08 | Manejo de errores y continuidad. Si la API o la base de datos no están disponibles, el frontend debe informar al usuario mediante un mensaje comprensible, evitar que la interfaz quede bloqueada y permitir las consultas contempladas en el respaldo de localStorage. | Fiabilidad | Esencial |
| RNF-09 | Mantenibilidad del código. El frontend, el backend y el acceso a datos deben mantenerse organizados en módulos o componentes con responsabilidades diferenciadas. Las instrucciones para instalar, configurar y ejecutar el proyecto deben estar documentadas en el README. | Mantenibilidad | Deseable |
| RNF-10 | Verificación y pruebas. Las funcionalidades principales deben contar con pruebas automatizadas o casos de prueba reproducibles. La ejecución de npm test debe finalizar e informar claramente los casos aprobados y fallidos, sin requerir servicios de pago reales. | Pruebas | Esencial |

*Nota de alcance: estos requisitos describen objetivos verificables para el entorno académico de desarrollo y demostración. No implican que la versión actual cuente con autenticación y autorización robustas en la API, alta disponibilidad productiva ni procesamiento de pagos reales; esas capacidades están fuera del alcance o figuran como mejoras futuras en este ERS.*